JISSec
Journal of Information System Security

A HIVE STREAM ENCRYPTION (HSE): A NEW STREAM ENCRYPTION ALGORITHM

Mahmoud Barnawi[1], Saad Alajmi[1], and Bin Mai[2]

[1]Hives System Encryption, LLC, USA; [2]Texas A&M University, USA

Journal of Information System Security is a publication of the Information Institute. The JISSec mission is to significantly expand the domain of information system security research to a wide and eclectic audience of academics, consultants and executives who are involved in the management of security and generally maintaining the integrity of the business operations.

Editor-in-Chief
Gurpreet Dhillon
The University of North Carolina, Greensboro, USA

Managing Editor
Filipe de Sá-Soares
University of Minho, Portugal

Publishing Manager
Mark Crathorne
ISEG, Universidade de Lisboa, Portugal

ISSN: 1551-0123
Volume 14, Issue 2

www.jissec.org

Abstract

In this paper, we present a new encryption algorithm, Hive Stream Encryption (HSE), based on stream encryption techniques. The proposed algorithm offers the strength and complexity to overcome the weaknesses and shortfall in current stream encryption methods. We present a new symmetric encryption model that encrypts/decrypts information along a path through multi-dimensional numeric space composed of cells arranged to form one encryption map. The main idea is to eliminate any mathematical relationship between the path and the encryption values in the cells in order to eliminate most of the issues inherited in the current classic stream encryption methods.

We demonstrate that in our proposed algorithm, the time it takes for a brute-force approach to identify the encryption path increase exponentially with the number of cells in the HSE map, and the number of possible key combination also increases with the map size and key size, unlike the traditional stream encryption such as AES. It illustrated that HSE is effective and efficient for data encryption.

Keywords: Stream encryption, Hive Stream Encryption, encryption path, encryption.

JISSec
Journal of Information System Security

Introduction

The worldwide proliferation of large quantity of data transmitted in the open information and communication technology (ICT) infrastructure, especially the Internet, has thrust information security to the forefront of attention from the academic researchers, industry practitioners, as well as government agencies around the globe (e.g., Appari and Johnson, 2010; Halperin and Backhouse, 2008; Dlamini et al., 2009). Various legislative bodies have stipulated the protection of sensitive data (e.g., the HIPPA regulation for health related data, and the Sarbane-Oxley legislation for financial data). Researchers have developed a wide variety of technical (e.g., Ahmad et al 2012; Ford and Baum 2000; Scarfone et al. 2008), behavioral (e.g., Crossler et al., 2013; Mai et al., 2017; Herath and Rao, 2009), and economic (e.g., Anderson and Moore, 2006; Jerman-Blazic, 2008; Mai et al., 2016) tools and mechanisms to ensure information security, and business organizations have embraced the challenge of information security and have heavily invested to improve the security of their business information.

Among the different information security tools and mechanisms available, encryption is one of the most fundamentally established and universally adopted systems used to improve information security (e.g., Rogaway and Coppersmith, 1998; Wang and Yu, 2009; Bosch et al., 2015; Schneier, 1993; Zhang et al. 2005). The continuous advancement of information and communication technology had enabled the development of more and more advanced encryption systems.

Generally speaking, encryption algorithms can be classified into two types: symmetric key encryption (i.e., private key encryption) and asymmetric key encryption (i.e., public key encryption). Symmetric key encryption has the significant advantage of high efficiency of implementation, but faces a fundamental disadvantage of key management necessary for secure implementation. To address this disadvantage, asymmetric key encryption has been developed. While asymmetric key encryption has successfully addressed the key distribution issue of symmetric key encryption, the implementation of asymmetric key encryption is significantly less efficient. As a result, today's encryption systems are mostly a hybrid system (Mao 2003, 2006; Stallings, 2006), in which symmetric key encryption is used to encrypt the communication messages, while the related private key used is encrypted using asymmetric key encryption.

Symmetric key encryption is implemented as either block encryption or stream encryption, the former of which encrypts inputs in blocks of plaintext, and the latter of which encrypts inputs by individual characters.

Modern stream encryption is based on one model by which the plain text is logically combined with a pseudorandom key-stream on bits level (e.g., Daemen and Clapp 1998; Min and Chen, 2013). However, due to many weaknesses resulted mostly from the mathematical dependencies of the model, stream encryptions have never achieved the level of reliability and confidence that would allow them to be applied more extensively in modern computing (e.g., Daemen et al., 1993).

All modern stream encryption systems are based on the famous "One-Time-Pad" (OTP) encryption (Hammersmith 2002) by which the plain text is encrypted letter by letter using a key-stream that is of the same size as the plain text through direct mapping. The key-stream is generated randomly and only used once and has no mathematical relation neither with the plain text nor the outcome, which makes the "one-pad" encryption theoretically mathematically unbreakable. However, this comes with significant implementation requirements that make it expensive to implement and very limited to high end application (e.g., Rueppe,l 1986).

Therefore, the OTP concept was implemented differently by which the key-stream is generated one a pseudorandom manner through a logical function and combined bit-by-bit with the plain text using logical operation such Exclusive OR (XOR). Due to this logical dependency, modern stream encryptions have inherited vulnerabilities such as repetitive key-stream, encryption key can't be re-used and detectable mathematical functions. All these limitation has made cipher encryption breakable and unreliable for heavy applications. On the other hand, block cipher offers much stronger encryption however due its bulky nature that is based on encrypting big chunks of data at once, it has limited application in real-time and stream communications. Hence, developing a new stream encryption that offers "one-time-pad" strength and efficiency would enable many new and reliable real time applications for encryptions.

We present the design of a new stream encryption method that offers the strength and complexity to overcome the weaknesses and shortfall in current stream encryption methods using a new symmetric encryption model that encrypts/decrypts information along a path through multi-dimensional numeric space composed of cells arranged to form one encryption map. The main idea is to eliminate any mathematical relationship between the path and the encryption values in the cells in order to eliminate most of the issues found in the current classic stream encryption methods. Unlike modern stream encryption, the present invention uses the 2D/3D & 4D numeric map as an encryption function and the encryption path as a key-stream. The path is generated from a symmetric secrete encryption key shared between the sender and the receiver, who both share the same encryption map with its

parameters. The encryption map parameters has to be designed to yield complexity that would make breaking the encoding computationally infeasible even using today's fastest supercomputers. Another fundamental difference between present invention and current stream encryption methods is that encryption/decryption process is taking place on the Unicode characters hexadecimal values (4-bits at a time) instead of 1-bit at time, which makes the encryption process more complex to break against traditional attacks yet faster and more efficient.

The rest of the paper is organized as follows. In section 2, we describe how HSE works in details and illustrate how it can be incorporated in a secured encrypted communication protocol. In section 3, we provide an analysis of its performance in terms of encryption speed and security. And finally in section 4, we conclude and point out future research direction.

Algorithm Description

The proposed encryption algorithm can be visualized by a map of hexadecimal numbers arranged into encryption cells. Figure 1 shows an example of such an encryption cell that makes up the Hive encryption map. As shown in the Figure 1, each cell (denoted by its cell index (1)) is of a hexagon shape contains 16 Encryption Point (2) along its 6 sides. Each Encryption Point (2) has two values; the Plain Text Value (3) and the Cipher Text Value (4). The present encryption is Hexadecimal based where encryption is taking place 4-bits at a time instead of bit by bit basis.

At an Encryption Point the encryption process takes place by direct mapping of the hexadecimal number represented in the Plain Text Value (3) with the corresponding hexadecimal number represented in the Cipher Text Value (4).

In actual implementation, there will be a Hive encryption map consisting of multiple encryption cells like the one depicted in Figure 1. For example, Figure 2 illustrates a Hive encryption map that is composed of 12 encryption cells. The map is constructed such that the Cipher Text Values within the Encryption Cells can either be generated randomly (Confusion) or by constantly shifting these values, within copies of the randomly generated cells, one step in clock-wise direction (Diffusion). For example the Encryption Cell number 12 is generated randomly and Encryption Cell number 1 the Cipher Text Value is the same value as Encryption Cell number 12 but shifted one step clock-wise. So in Cell number 12 the Plain Text Value of 0x0 is encrypted by 0xA of Cipher Text Value, however in cell number 1 0x0 is encrypted by 0x6 and so on. Also Figure 2 shows the Encryption Path (4) in the map, which represents the order of cells the encryption process will take place in, and in this case it is 1→2→7→3→4→9→8→5→6→11→10→12.

JISSec
Journal of Information System Security

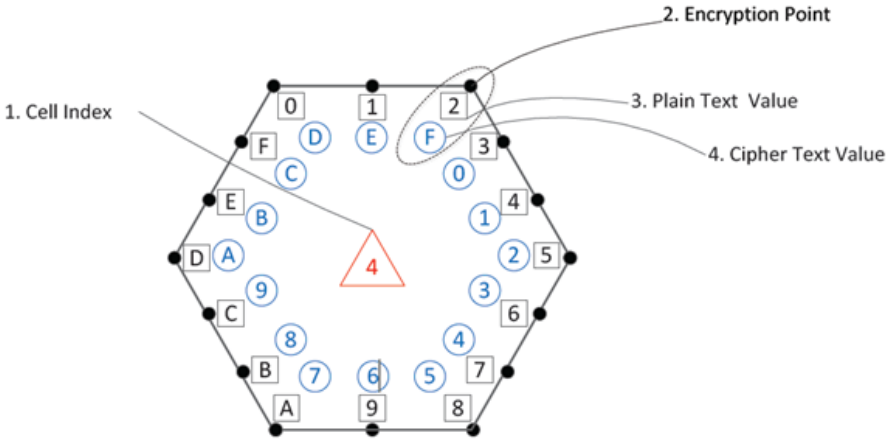


Figure 1: Conceptual representation of an Encryption Cell in the Hive Stream Encryption Map

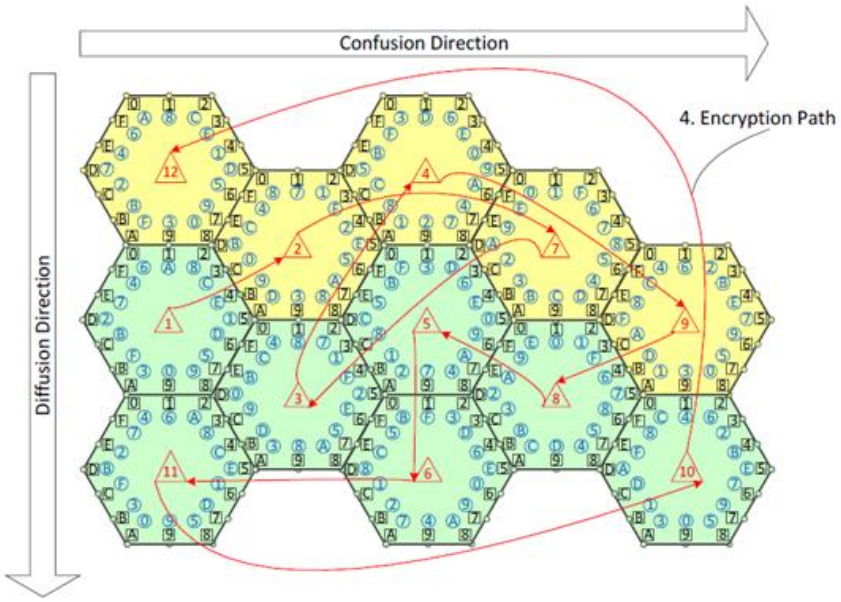


Figure 2: A Hive Encryption Map showing the encryption cells and the encryption path

According to the encryption path on the Hive map shown in Figure 2, the word "THE" can be encrypted as illustrated in the following Table 1.

Plain Text (Character)	T				H				E			
Plain Text (Unicode Hex)	0x0	0x0	0x5	0x4	0x0	0x0	0x4	0x8	0x0	0x0	0x4	0x5
Encryption Path (Cells)	1	2	7	3	4	9	8	5	6	11	10	12
Cipher Text (Hex)	0x6	0x8	0x8	0xF	0x3	0x4	0x6	0x4	0xB	0x4	0xB	0xD

Table 1: Encrypting the word "THE" using Hive map and path shown in Figure 2

If we convert the above encryption operation in Table 1 into binary it will yield the following:

Input:	0000	0000	0101	0100	0000	0000	0100	1000	0000	0000	0100	0101
	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
Output:	0110	1000	1000	1111	0011	0100	0110	0100	1011	0100	1011	1101

As per the encryption operation in Table 1, the following can be noticed:

1. There is no mathematical relationship between the Plain Text values and the Cipher Text values;
2. There is no mathematical relationship between the encryption path and neither Plain Text values nor the Cipher Text values, as the path is governed by an independent mechanism/algorithm;
3. The encryption process is taking place on hexadecimal number (that's 4-bits a time), which makes Hive Encryption 4-times faster than current stream encryption methodologies;
4. Each Plain Text Hexadecimal requires one cell hence the encryption path has to be long enough for encrypting all Plain Text data.

As can deduced from the above description, the Hive encryption process is very straight forward and based on mapping and doesn't involve complex mathematical calculations, which makes it easy to implement with the right protocol and measurements in place.

The Hive Encryption map has the following properties:

Scalable Complexity: the complexity of the map can be scaled and managed easily as it's a matter of increasing the number of the encryption cells. The complexity of map is measured by the possible number of path it contains which can be calculated as the following example:
Number of Possible Encryption Path = 2^n, 3^n, 4^n, where n is number of encryption cells. So for example for a 2,000 cells map, the number of possible encryption path will be $2^{2,000} = 1.2 \times 10^{602}$ unique paths capable of encrypting more than 2.2 $\times 10^{587}$ TB of data before the path repeats itself again.

Confusion and Diffusion: by either generating the Cipher Text Values randomly (Confusion) or by rotating them the Cipher Text Value of the randomly generated Encryption Cells one step clockwise (Diffusion). The resulted cipher will be scrambled and looks totally random from external attacker's point of view.

Encryption Path Generation Function: the encryption key is used as a seed for generating the required path for encrypting the plain text. This function is unique and its consistence is such that when the sender and the receiver use two copies of the same map and the symmetric encryption keys the generated path will be the same at both ends.

Encryption Map Tuning Function: In order to ensure each encryption session is unique and will generate different set of encryption data the map has to be tuned each time it's used. To implement that the map will use Encryption Map Tuning Function, by which the cells within the map can be re-arranged accordingly. This function enables changing and scrambling the values of the map to create a new topology of the map to enable the re-use of the same map between the same users more than one time.

The Encryption Path is generated from the cells' numbers according to a function that associated with the Hive map. This function is responsible for generating the Encryption Path using the encryption key that will be shared between the sender and receiver. To achieve this, the encryption key itself is a subset of the encryption path and represents the first part of the path.

A Hive encryption protocol would operate as the following steps:

1. The process starts by the sender and the receiver getting copy of a common Hive map they will use for the encryption / decryption of data between them. This common Hive map will be standard and would be found on the World Wide Web and come with different sizes and Encryption Path Function;

2. The sender then, for the first time during the setup of the encryption, will tune the map using set of define parameters or initiation vector in order to customize it for the communication with the receiver;

3. The sender then will send the tuning parameters or initiation vector to the receiver;

4. The receiver will in turn tune the map to be identical to the sender's map;

5. The sender then generates from the map an Encryption Key that will be used symmetrically with the receiver for the encryption process. The size of the key can vary depending on the agreed parameters between the sender and receiver;

6. After generating the encryption key the sender will then send it to the receiver over a secure channel using any of the known standard methods for exchanging secret symmetric keys;

7. The sender then will use the encryption key along with the map to encrypt the plain text he/she would like to send. The map will work on the same dynamic explained above by taking the encryption key as a seed to generate the required length of the encryption path in accordance with the map's path function as explained using Figure 3.

8. The encryption process takes place and the cipher text is streamed down the (unsecure) communication channel between the sender and the received. Given the dynamic of the Hive encryption, there is no limitation in the length of the encryption process that will go on as long as it's needed.

9. The receiver will then uses the Hive map and the encryption key to decrypt the cipher text into the original plain text that was sent.

The above steps can be illustrated by the following Figure 3.

JISSec
Journal of Information System Security

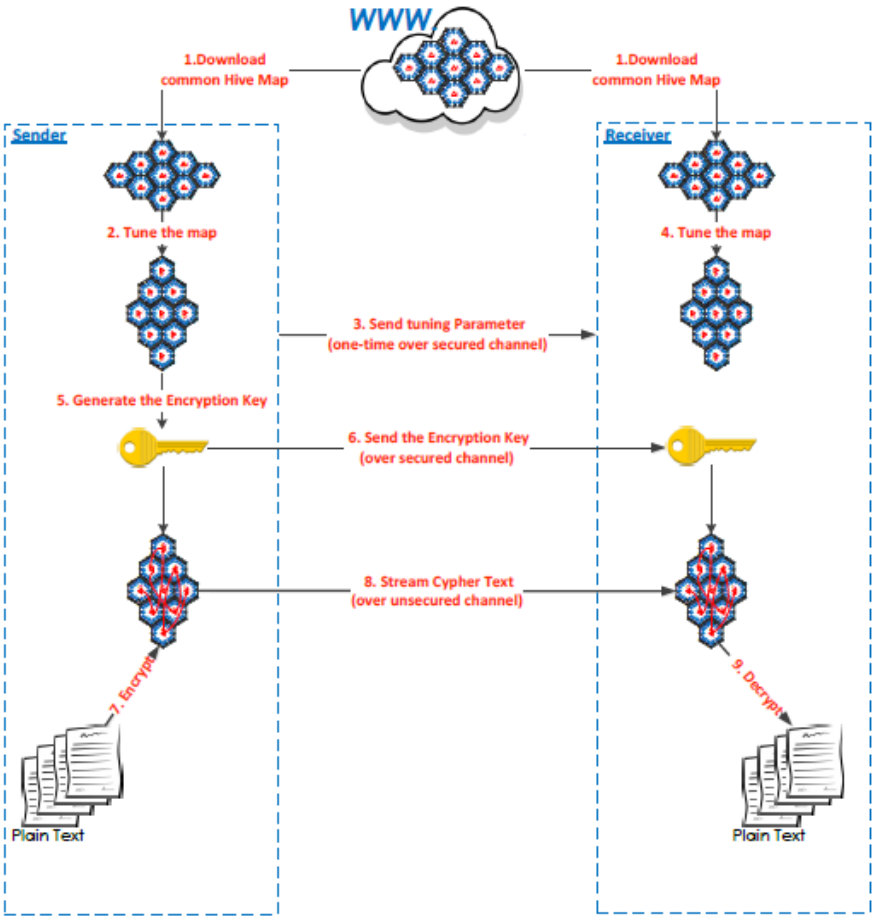


Figure 3: Hive Encryption Protocol for encrypting and decrypting information

In terms of implementation, the proposed algorithm can be programmed using structured programming language (such as C) or object oriented language (such as C++ or C#) as the following brief design outline:

1. Encryption Cells can be programmed as user-define structure type as the following:

```
Struct EncryptionCell {
Int CellNumber;
Unsigned char plainTextValues [16];
Unsigned char cipherTextValue [16];
};
```

2. The Hive map can be coded as one dimensional array or *EncryptionCell* type as the following:

   EncryptionCell HiveMap [MAP_SIZE];

3. The encryption process can be initiated by creating the encryption path using the encryption key as the following:

   CreateEncryptionPath (EncryptionKey key1);

4. After the generation of the encryption path the process can start as the following:

   While (plainTextHexChar = getCharFromStream() !=EndOfStream){

   cipherTextHexChar = HiveMap [plainTextHexChar];

   sendCipher (cipherTextHexChar);

   moveToNextEncryptionCellOnPath();

   };

5. The decryption will the reverse of the encryption process and can be programmed in the same way.

To enable the re-use of the encryption key this invention uses two mechanisms:

6. Use of time stamp that can be used in conjunction with the encryption key to generate a unique encryption path depending on the time of the communication session;

7. Use dynamic cell padding by assigning random values to the outcome produced by some selected encryption cells that yield different stream cipher if the key is re-used.

Encryption System Design and Performance

We construct a prototype of the encryption system with the following design criteria:

#	Criteria	Size	Complexity
1	Map Size	32,000 cell	$2^{32,000} = 9\times10^{9,632}$ (possible unique encryption path)
2	Encryption Key Size	64 cell	$32,000^{64} = 2\times10^{288}$ (possible keys)

With the above design criteria the strength of the systems are analyzed as follows:

- Size of data that can be encrypted before the encryption path is repeated:

  Assuming each path is capable of encrypting one Hexadecimal:

  - $9\times10^{9,632}$ Hexadecimal $\rightarrow 9\times10^{9,632} \times 16 = 1\times10^{9,634}$ bits = $1.8\times10^{9,615}$ Exabyte of information before the path repeats itself (1 Exabyte = 1018 Byte)
- In order to compute all possible paths of the map using Brute-Force attack using the most powerful available supercomputer nowadays capable of executing 33.86×10^{15} Floating-Point Operations Per Second (FLOPS) and assuming a path is calculated per each FLOPS the time needed is as the following:
  - $9\times10^{9,632} / 33.86\times10^{15} = 2.7\times10^{9,616}$ seconds = $8.5\times10^{9,608}$ years, which is computationally unfeasible
- Using similar calculation, in order to break the encryption key using Brute-Force attack using the most powerful available supercomputer nowadays assuming a possible key combination is calculated per each FLOPS, the time needed to try all combinations is the following:
  - $2\times10^{288} / 33.86\times10^{15} = 6\times10^{271}$ seconds = 2×10^{264} years, which is computationally unfeasible

The memory footprint of the design: 32,000 × ((16 × 8) + 16) = 4,608,000 bits = 576Kbyte.

Figure 4 below illustrates how much time it would take before encryption path is repeated against different map sizes. We use log scale for the time length on the y-axis.

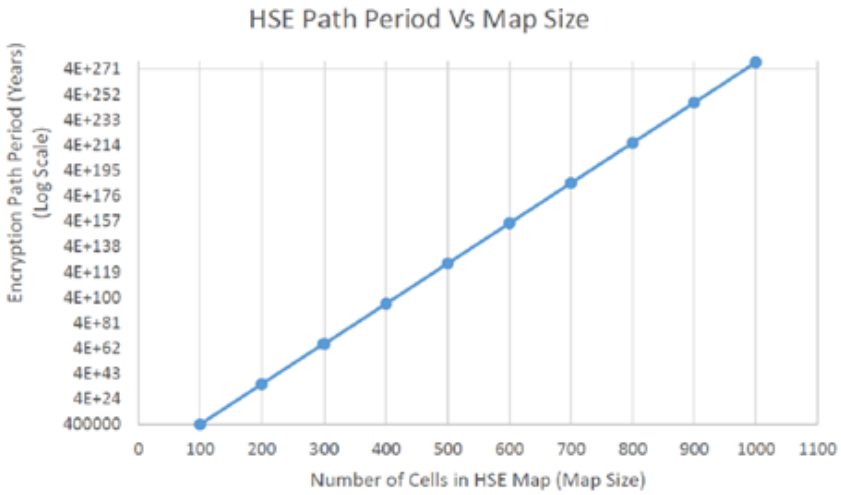


Figure 4. The Relationship between the Encryption Path Repetition Period Length and Encryption Map Size

Figure 5 shows how the complexity of the encryption key (i.e., the number of possible key combinations) increases with the key size.

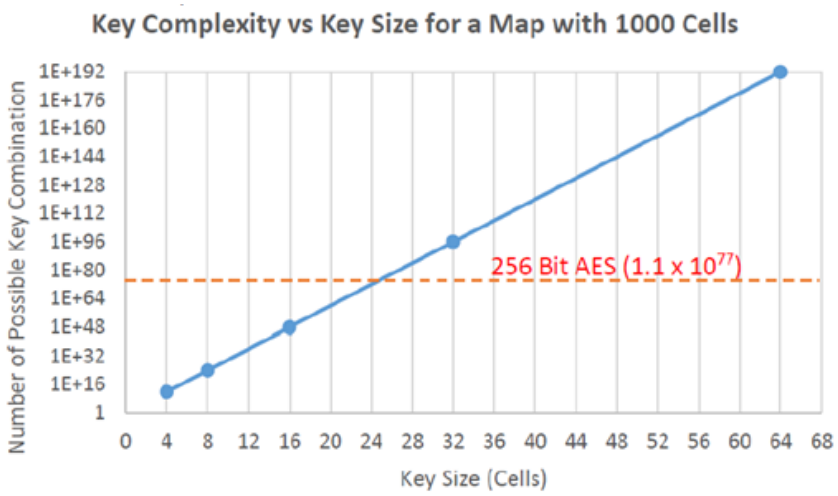


Figure 5. The Relationship between the Encryption Key Complexity and Encryption Key Size

And Figure 6 shows how Key complexity increases with the encryption map size for a fixed Key Size of 64 cells.

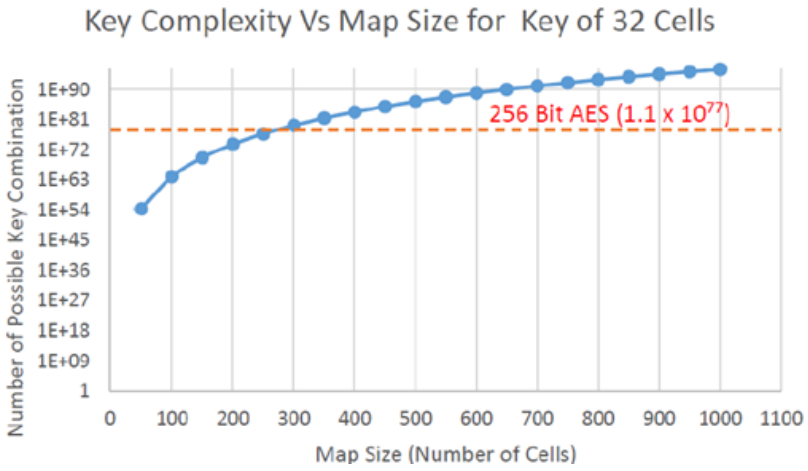


Figure 6. The Relationship between the Encryption Key Complexity and Encryption Map Size

JISSec
Journal of Information System Security

Table 2 shows the resistance to modern stream cipher attacks of the invention:

#	Attack	Description	Impact on Hive Encryption
1	Exhaustive Search (Brute Force Attack)	Exposing the inner state of the systems such as encryption key or encryption functions by applying and analyzing all possible input.	Given the complexity of the present invention and design, theoretically it would take more than $8.5\times10^{9,608}$ year to discover the encryption function and more than 2×10^{264} years to break the key using Brute Force Attack using the power of the most powerful supercomputers available nowadays that can do up to 33.86×10^{15} Floating-Point Operations Per Second (FLOPS)
2	Re-used Key Attack	Extracting the plain text by sending two messages A and B using the same key and through working out the logical relationship between the key and the cipher of the two messages using XOR function.	No impact as there is no mathematical relationship between the cells in the encryption key/path.
3	Bit-Flipping (Fault Attack)	Altering the content of the plain text by combining known part of the cipher text with a new message using XOR relationship to yield a desired outcome	No predictable result can be achieved as there is no relationship between the plain text and cipher text, which removes any controlled impact. However this can result in disrupting the plain that can result in a general Denial of Service type of attack.
4	Algebraic Attack	Used in stream cipher that is based on Linear Feedback Shift Registers (LFSR) and it aims at finding and resolving set of low degree equation derived from the key and the cipher in order to expose the key	No impact as there is no mathematical relationship between the cells in the encryption key/path.
5	Correlation Attack	Used in stream cipher that is based on Linear Feedback Shift Registers (LFSR) and it aims at exposing any statistical dependencies between the key and the cipher text.	No impact as there is no statistical relationship between the cells in the encryption key/path.
6	Distinguishing Attack	Using a linear function to distinguish the cipher distribution from the random distribution.	Further research might be needed to determine the impact
7	Chosen-IV Attack	This attacks aims at extracting the Initialization Vector from the memory of Linear Feedback Shift Registers (LFSR) in order to exploit the weakness in the key scheduling algorithm by finding and resolving system of equations.	No impact as there is no mathematical relationship between the cells in the encryption key/path.
8	Slide Attack	Uses possible relationship in the cipher text by comparing the outcome of two encryption runs at one step shift between the two outcomes.	No impact as there is no mathematical relationship between the cells in the encryption key/path.
9	Cube Attack	Discovers any low degree polynomial between of the output as a function of the key and the plain text in order to derive the key.	No impact as there is no mathematical relationship between the cells in the encryption key/path.
10	Guess and Determine Attack	Guesses as set of the state elements of the cryptosystem and compares it with resulting key sequences with complexity up to 2^{224}	Limited and manageable Given the complexity of the present invention and design, theoretically it would take more than $8.5\times10^{9,608}$ year to discover the element of the cryptosystem using the power of the most powerful supercomputers available nowadays.

Table 2: Common Cipher Attacks and their Impacts on the Hive Encryption System

The Hive map can be configured to 3 special dimensions as can be shown in Figure 7. In this figure the Cells composing the map are arranged along the X, Y and Z axis. In this configuration the encryption path can travel across the three axes. This configuration allows higher level of complexity and increasing the confusion and diffusion level in the map.

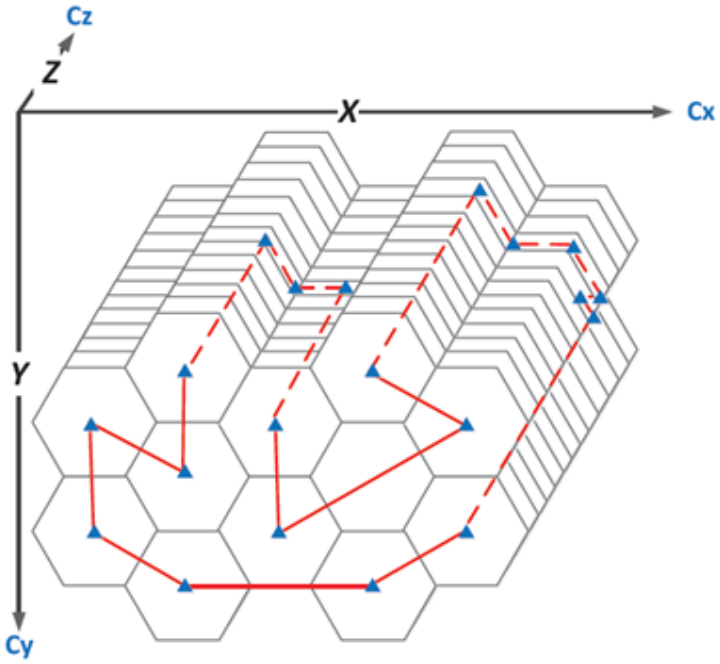


Figure 7: Conceptual model of 3D Hive Space with encryption path propagating in all three dimensions

As per this compilation the number of Cells within the map is equal to:

$Number\ of\ Cells = Cx \times Cy \times Cz$

Where Cx is number of Cells along X axis, Cy is the number of Cells along Y axis and Cz is the number of Cells along Z axis.

The number of possible encryption path is equals to:

$Number\ of\ encryption\ path = 2^{number\ of\ cells}$

In terms of implementation the 3D can be programmed in the same way as the 2D map except with the following differences:

- The map is represented by three dimensional array declared as the following

  EncryptionCell HiveMap[Cx] [Cy] [Cz];

  Where Cx, Cy and Cz are integers represents number of Cells in each axis
- The path is generated by mathematical manipulation of the indices of the map's array within the Path Generation Function

In addition to three-dimensional configuration, the Hive map can be configured for a four or more dimensions, three spatial and one time so that the encryption will depends on both the encryption path and on the time of the encryption. This can be implemented by enabling the map to continuously change over time by shifting of the Cells location and/or the encryption value within the Cells. This configuration will allow for real-time and non-reversible map encryption ideal for a dedicated one-to-one communications session between two end users.

To ensure correctness of encryption process, the following should be considered:

- Whenever the sending map undertakes any timely change the receiving map must be informed in order to undertake the same change. This can happen through exchanging defined encrypted messages in the middle of the encryption process
- The change to Hive map will undertake is irreversible
- In addition to an Encryption Path Generation function, the system must implement a Map Evolution Function to be one of the Hive Map's properties.
- In order to make the Map Evolution Process unique to each communication session it can be based on initial parameter derived from a common information between the sender and the receive (for example the Encryption Key)

The Hive map can be configured to more than 4-dimensions in terms of time and space however this will require a programming language that supports more than 3 dimensional array manipulations. In this configuration, the Hive map will take the shape of conceptual 4D shapes such as tesseract or a hypersphere supported by modern mathematical models of such shapes. However, the map will work on exactly on the same principle as 2D or 3D map the only difference the Encryption Path Generation Function and the Map Evolution Function will have to be programmed for such hyper shapes.

Conclusion

In this paper, we propose a new stream encryption algorithm, Hive Stream Encryption (HSE) that improves upon the existing stream encryption methods. The core of HSE is a new symmetric encryption model that encrypts/decrypts information along a path through multi-dimensional numeric space composed of cells arranged to form one encryption map. We show that HSE is able to eliminate any mathematical relationship between the path and the encryption values in the cells and as a result, can eliminate most of the issues inherited in the current classic stream encryption methods.

The natural next step in this research, which we are undertaking right now, is to develop a prototype of the HSE system, and measure its performance in a real-world setting.

References

Ahmad, S. M. S., Ali, B. M., and Adnan, W. A. W. (2012). Technical issues and challenges of biometric applications as access control tools of information security. *International Journal of Innovative Computing, Information and Control*, *8*(11), 7983-7999.

Anderson, R., and Moore, T. (2006). The economics of information security. *Science*, *314*(5799), 610-613.

Appari, A., and Johnson, M. E. (2010). Information security and privacy in healthcare: current state of research. *International journal of Internet and enterprise management*, *6*(4), 279-314.

Bösch, C., Hartel, P., Jonker, W., and Peter, A. (2015). A survey of provably secure searchable encryption. *ACM Computing Surveys (CSUR)*, *47*(2), 18.

Crossler, R. E., Johnston, A. C., Lowry, P. B., Hu, Q., Warkentin, M., and Baskerville, R. (2013). Future directions for behavioral information security research. *Computers & Security*, *32*, 90-101.

Daemen, J., and Clapp, C. (1998). Fast hashing and stream Encryption with PANAMA. In *Fast Software Encryption* (pp. 60-74). Springer Berlin/Heidelberg.

Daemen, J., Govaerts, R., and Vandewalle, J. (1993, May). Resynchronization weaknesses in synchronous stream ciphers. In *Workshop on the Theory and Application of Cryptographic Techniques* (pp. 159-167). Springer, Berlin, Heidelberg.

Dlamini, M. T., Eloff, J. H., and Eloff, M. M. (2009). Information security: The moving target. *Computers & security*, *28*(3), 189-198.

Ford, W., and Baum, M. S. (2000). *Secure electronic commerce: building the infrastructure for digital signatures and encryption*. Prentice Hall PTR.

Halperin, R., and Backhouse, J. (2008). A roadmap for research on identity in the information society. *Identity in the information society*, *1*(1), 71-87.

Hammersmith, W. (2002). *U.S. Patent Application No. 10/254,743*.

Herath, T., and Rao, H. R. (2009). Encouraging information security behaviors in organizations: Role of penalties, pressures and perceived effectiveness. *Decision Support Systems*, *47*(2), 154-165.

Mai, B., Kulkarni, S., M. Salehan, "Vulnerability enhancement vs. loss mitigation: Optimal information security investment," Journal of Information Systems Security, Vol. 12, No. 2, 2016

Mai, B., Parsons, T., Prybutok, V., and K. Namuduri, "Neuroscience foundations for human decisionmaking in information security: A general framework and experiment design," in Davis, F., Riedl, R., vom Brocke, J., Leger, P., and A. Randolph, (Eds.) Information Systems and Neuroscience – Gmunden Retreat on NeuroIS 2016, Switzerland, Springer International Publishing, ISBN 978-3-319-41402-7, 2016.

Mao, W. (2003). *Modern cryptography: theory and practice*. Prentice Hall Professional Technical Reference.

Min, L., and Chen, G. (2013). A novel stream encryption scheme with avalanche effect. *The European Physical Journal B*, *86*(459), 1-13.

Rogaway, P., and Coppersmith, D. (1998). A software-optimized encryption algorithm. *Journal of Cryptology*, *11*(4), 273-287.

Rueppel, R. A. (1986). Stream ciphers. In *Analysis and Design of Stream Ciphers* (pp. 5-16). Springer Berlin Heidelberg.

Scarfone, K., Souppaya, M., Cody, A., and Orebaugh, A. (2008). Technical guide to information security testing and assessment. *NIST Special Publication*, *800*, 115.

Schneier, B. (1993, December). Description of a new variable-length key, 64-bit block cipher (Blowfish).

In *International Workshop on Fast Software Encryption* (pp. 191-204). Springer, Berlin, Heidelberg.

Stallings, W. (2006). *Cryptography and network security: principles and practices*. Pearson Education India.

Wang, X. Y., and Yu, Q. (2009). A block encryption algorithm based on dynamic sequences of multiple chaotic systems. *Communications in Nonlinear Science and Numerical Simulation*, *14*(2), 574-581.

Zhang, L., Liao, X., and Wang, X. (2005). An image encryption approach based on chaotic maps. *Chaos, Solitons & Fractals*, *24*(3), 759-765.